COMPUTER NETWORKS · SEMESTER 1 · LECTURE 5

The application layer III: electronic mail, Usenet, DNS and NAT

Week 5 teaching pack: lecture notes, seminar, Linux laboratory and Windows laboratory (two hours each)

Assoc. Prof. Nicolaie Popescu-Bodorin, PhD, 2026
+40722882471, bodorin(on)live.com, https://lmrec.org/bodorin/

Part	Contents	Duration
A	Lecture 5 — The application layer III: SMTP, mail format and MIME, POP3 and IMAP, Usenet, DNS and NAT	2 h
B	Seminar 5 — Messages, names and translated addresses	2 h
C	Laboratory 5 (Linux) — Mail by hand, DNS byte by byte over UDP, a NAT router of your own	2 h
D	Laboratory 5 (Windows) — The same protocols with Windows tools and Winsock; finding your own NAT	2 h

Part A · Lecture 5

After this lecture you should be able to

- name the components of the e-mail system and follow a message from sender to reader;
- read and write an SMTP dialogue, and distinguish the envelope from the message header;
- explain the format of a mail message and how MIME carries attachments and non-ASCII text;
- compare POP3 and IMAP, and say how mail is protected against eavesdropping and forgery;
- describe Usenet and how NNTP servers flood articles to one another;
- explain DNS: its services, its hierarchy, iterative and recursive resolution, caching, resource records and message format;
- explain how NAT works, why it exists, and which applications it breaks.

Plan of the two hours

Part	Topic	Time
Hour 1		
1	The last week of the application layer	3 min
2	Electronic mail: the system	7 min
3	SMTP: pushing mail	15 min
4	The message: format and MIME	10 min
5	Reading mail: POP3, IMAP and webmail; protecting mail	15 min

Part	Topic	Time
	Break	10 min
Hour 2		
6	Usenet: news by flooding	8 min
7	DNS: services and hierarchy	12 min
8	DNS: resolution, caching, records and messages	15 min
9	NAT: many hosts behind one address; summary	15 min

1. The last week of the application layer

Weeks 3 and 4 covered HTTP, HTTPS, TELNET, SSH and FTP. This week completes the syllabus list: the e-mail protocols **SMTP**, **POP** and **IMAP**, the news system **Usenet**, the **Domain Name System** on which almost every other protocol depends, and **network address translation**, which is not an application protocol at all but a network-layer function that every application protocol must live with. Strictly, NAT belongs with the network layer — we meet it now because of what it does to applications, and return to its mechanics there.

2. Electronic mail: the system

E-mail (1971) is older than TCP/IP and still one of the most used Internet applications. Unlike the Web, it is **asynchronous**: the reader need not be on line when the message is sent. Its main components are:

- **User agents** (mail programs) — Thunderbird, Outlook, the Mail app of a phone, or a webmail page — with which people write and read messages.
- **Mail servers**, each holding the **mailboxes** of its users and a **queue** of outgoing messages. A server that accepts a user's new message is a *submission server*; a server that relays mail between domains is a *mail transfer agent*.
- **Protocols**: **SMTP** carries a message from the sender's user agent to its server and from server to server — it *pushes*. **POP3** and **IMAP** let the reader's user agent fetch the message from the mailbox — they *pull*. The **DNS** tells a sending server which server receives mail for a domain.

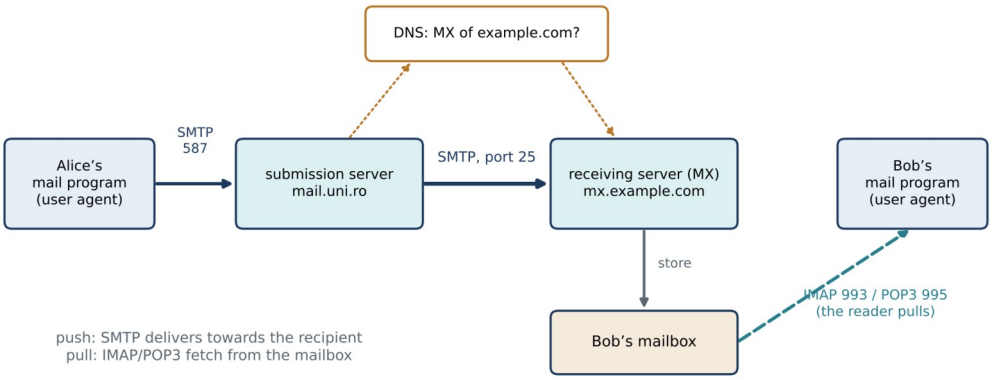


Figure 1. The path of a message. SMTP pushes it from Alice's user agent to her provider's server and from there to the server named in the recipient domain's MX record; Bob's user agent later pulls it with IMAP or POP3.

Why not send directly from Alice's laptop to Bob's? Because Bob's mailbox must be reachable when Bob is not, and Alice's laptop must be able to hand over the message and go off line: if the receiving server is down, Alice's server keeps the message in its queue and retries, typically every half hour for several days, before returning an error report to Alice.

3. SMTP: pushing mail

SMTP (Simple Mail Transfer Protocol; RFC 821, 1982; current version RFC 5321) runs over TCP. Like HTTP, it is a text protocol of commands and three-digit replies; unlike HTTP, the client pushes the message to the server, and several messages may follow one another on one connection. A complete dialogue, as you will type it in the laboratory:

```
S: 220 mx.example.com ESMTP ready
C: EHLO mail.uni.ro
S: 250-mx.example.com
S: 250-SIZE 52428800
S: 250 STARTTLS
C: MAIL FROM:<alice@uni.ro>
S: 250 OK
C: RCPT TO:<bob@example.com>
S: 250 OK
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: From: Alice <alice@uni.ro>
C: To: Bob <bob@example.com>
C: Subject: Lab 5
C:
C: Hello Bob, see you on Monday.
C: .
S: 250 OK: queued as 7F3A2
C: QUIT
S: 221 Bye
```

Command	Meaning
`HELO` / `EHLO name`	greeting; EHLO (extended SMTP) asks the server to list its **extensions**: SIZE, 8BITMIME, STARTTLS, AUTH, SMTPUTF8, …
`MAIL FROM:<address>`	starts a transaction; the **reverse path**, to which error reports (bounces) go
`RCPT TO:<address>`	one recipient; repeated for each recipient on this server
`DATA`	the message follows, ended by a line holding a single dot

Command	Meaning
`RSET`, `NOOP`, `QUIT`	abort the transaction; do nothing; end the session
`STARTTLS`, `AUTH`	switch the connection to TLS (RFC 3207); authenticate the user (RFC 4954)

Replies follow the scheme of FTP: 2xx success (220 ready, 250 OK, 221 closing), 3xx send more (354 start mail input), 4xx temporary failure — try again later (421, 450 mailbox busy, 451), 5xx permanent failure (550 no such user, 552 too large, 554 rejected).

3.1 Three details that matter

- **Framing.** The message ends at a line containing only a dot. A line of the message that begins with a dot is sent with an extra dot (**dot-stuffing**), which the receiver removes — the delimiter method of Lecture 3, with escaping.
- **Envelope and header are different.** `MAIL FROM` and `RCPT TO` form the **envelope**, used for delivery, like the address on a paper envelope. The `From:` and `To:` lines inside the message are the **header**, shown to the reader, like the letterhead. They may differ: a Bcc recipient appears in the envelope but in no header line; a mailing list delivers to thousands of envelope addresses a message whose `To:` names the list. And nothing in plain SMTP checks either — which is why forged senders exist (section 5.3).
- **Ports. 25** for server-to-server relay; **587** for submission by user agents, after authentication (RFC 6409); **465** for submission over TLS from the first byte (RFC 8314). Most ISPs block outgoing port 25 from home connections, so that infected PCs cannot send spam directly.

3.2 Finding the receiving server

Alice's server must find which server receives mail for *example.com*. It asks the DNS for the domain's **MX** (mail exchanger) records, for example `example.com. MX 10 mx1.example.com.` and `MX 20 mx2.example.com.`, tries the server with the lowest preference number first and falls back to the others. This is our first example of an application that cannot work without DNS.

3.3 SMTP and HTTP compared

	HTTP	SMTP
Direction	pull: the client asks for an object	push: the client delivers a message
Connection	persistent (HTTP/1.1)	persistent: several messages per connection
Encoding	any bytes in the body (Content-Length)	originally 7-bit ASCII lines; binary needs MIME encoding or the 8BITMIME extension
Several objects	one response per object	all parts in one multipart message
Replies	three-digit status codes	three-digit reply codes

4. The message: format and MIME

SMTP only carries the message; its format is defined separately in **RFC 5322**: lines of text ending in CR LF, a **header** of `Name: value` fields, an empty line, and a **body**. The main header fields are `From`, `To`, `Cc`, `Subject`, `Date`, `Message-ID` (a worldwide unique identifier), `Reply-To`, `In-Reply-To`

and `References` (which link the messages of a conversation). Every server that relays the message adds a **Received** field at the top, so that the header records the message's path, most recent hop first.

RFC 5322 allows only ASCII text. **MIME** (Multipurpose Internet Mail Extensions, RFCs 2045–2049) adds everything else with a few more header fields:

Field	Meaning
`MIME-Version: 1.0`	the message uses MIME
`Content-Type`	the media type: `text/plain; charset=utf-8`, `text/html`, `image/jpeg`, `application/pdf`, or `multipart/mixed; boundary="…"` for several parts — the same media types HTTP uses
`Content-Transfer-Encoding`	how the bytes are written as text: `7bit`, `8bit`, `quoted-printable` (for mostly-ASCII text: =C8 for a byte 0xC8) or **`base64`** (for binary data)
`Content-Disposition`	`inline` or `attachment; filename="report.pdf"`

```
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary="==B7f2"

--==B7f2
Content-Type: text/plain; charset="utf-8"

Hello Bob, the report is attached.
--==B7f2
Content-Type: application/pdf
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="report.pdf"

JVBERi0xLjcKJcfsj6IKNSAwIG9iago8PC9MZW5ndGggNiAwIFIvRmlsdGVy...
--==B7f2--
```

base64 writes every 3 bytes as 4 characters from a 64-letter alphabet (A–Z, a–z, 0–9, +, /), in lines of at most 76 characters. An attachment therefore grows by a third, plus the line ends — about **37 %**: a 10 MB file becomes a message of about 13.7 MB, one reason mail servers limit message sizes. Non-ASCII text in header fields, such as a subject in Romanian, is written as an **encoded word** (RFC 2047): `Subject: =?utf-8?b?TGFib3JhdG9ydWwgNQ==?=`.

5. Reading mail: POP3, IMAP and webmail

SMTP cannot be used to read mail: it pushes, and the reader's computer is often off line or behind NAT. A reader's user agent therefore *pulls* messages from its mailbox with an access protocol.

5.1 POP3

POP3 (Post Office Protocol version 3; RFC 1939; port 110, or 995 over TLS) is deliberately simple. A session goes through three states:

- **AUTHORIZATION**: `USER bob`, `PASS secret` (each answered `+OK` or `-ERR`).
- **TRANSACTION**: `STAT` (number and total size of messages), `LIST` (size of each), `UIDL` (a unique identifier per message), `RETR n` (the message, ended by a line with a single dot, with dot-stuffing as in SMTP), `TOP n k` (header and k lines), `DELE n` (mark for deletion), `RSET` (unmark).

- **UPDATE**: after `QUIT`, the server removes the messages marked for deletion.

POP3 suits one device that downloads the mail and keeps it — *download and delete*, or *download and keep* with UIDL to avoid downloading twice. It knows nothing about folders, read flags or other devices.

5.2 IMAP

IMAP (Internet Message Access Protocol; IMAP4rev1 RFC 3501, IMAP4rev2 RFC 9051; port 143, or 993 over TLS) keeps the mail **on the server**, organised in **folders**, with per-message **flags** (`\Seen`, `\Answered`, `\Flagged`, `\Deleted`). Every device sees the same state, which is what people with a phone, a laptop and a webmail account expect. The client can search on the server and fetch only what it needs — the headers of a folder, one part of one message — useful on a slow mobile link. Commands carry a **tag** chosen by the client, so that several may be outstanding; the server sends untagged data lines (*) and ends each command with a line carrying its tag:

```
C: a1 LOGIN bob secret
S: a1 OK LOGIN completed
C: a2 SELECT INBOX
S: * 3 EXISTS
S: * 1 RECENT
S: a2 OK [READ-WRITE] SELECT completed
C: a3 FETCH 3 (FLAGS BODY.PEEK[HEADER.FIELDS (FROM SUBJECT)])
S: * 3 FETCH (FLAGS () BODY[HEADER.FIELDS (FROM SUBJECT)] {52}
S: From: Alice <alice@uni.ro>
S: Subject: Lab 5
S: )
S: a3 OK FETCH completed
C: a4 STORE 3 +FLAGS (\Seen)
C: a5 LOGOUT
```

The `{52}` announces that 52 bytes of literal data follow — a length prefix inside a line-based protocol. With the `IDLE` command (RFC 2177) the server can notify the client of new mail at once instead of being polled.

5.3 Webmail, and protecting mail

With **webmail** (Gmail, Outlook.com, a university's webmail) the user agent is a web page: the browser talks HTTPS to the webmail server, which uses SMTP to send and reads the mailbox itself; mobile apps often use proprietary protocols or **JMAP** (RFCs 8620–8621), a modern JSON-over-HTTPS replacement for IMAP.

Mail needs two kinds of protection. Against **eavesdropping**: user agents use TLS on ports 587/465/993/995, and servers upgrade their port-25 connections with STARTTLS — but this protects each hop, not the message; servers along the way read it in clear. Only end-to-end encryption by the user agents (**S/MIME** or **OpenPGP**) hides it from the servers. Against **forgery**: plain SMTP lets anyone write any `MAIL FROM` and `From:`; the receiving server checks three records published in the sender domain's **DNS**:

Mechanism	What the receiver checks
SPF (RFC 7208)	a TXT record listing the addresses allowed to send mail for the domain: `v=spf1 ip4:193.231.0.0/16 include:_spf.google.com -all`
DKIM (RFC 6376)	a digital signature over the header and body, added by the sending server in a `DKIM-Signature` field; the public key is in DNS

Mechanism	What the receiver checks
DMARC (RFC 7489)	a policy record at `_dmarc.domain`: the `From:` domain must match a passing SPF or DKIM check, otherwise quarantine or reject — and send reports

Since 2024 the largest mailbox providers reject bulk mail that does not pass these checks. Spam filtering adds content analysis and reputation on top.

6. Usenet: news by flooding

Usenet (1979–80; Tom Truscott and Jim Ellis at Duke, Steve Bellovin at UNC) is a worldwide distributed discussion system, older than the Web. Messages, called **articles**, are posted to **newsgroups** organised in hierarchies — the "Big Eight" `comp.`, `sci.`, `rec.`, `soc.`, `talk.`, `news.`, `misc.`, `humanities.`, plus `alt.` and many national and local ones: `comp.os.linux.networking`, `sci.physics`, `ro.comp.retele`. An article has the format of a mail message (RFC 5536) with extra header fields: `Newsgroups:`, a unique `Message-ID:`, `References:` (the thread) and `Path:`.

Usenet has **no central server**. Each site's news server keeps copies of the articles of the groups it carries, and exchanges new articles with its neighbouring servers by **flooding**: a server offers each new article to every peer, which accepts it only if it has not seen the `Message-ID` before. Each server that relays the article adds its own name to the left of the `Path:` field (`Path: news.uni.ro!feed.example.net!origin.example.com!not-for-mail`), and does not offer it to servers already named there. Within hours an article reaches thousands of servers, along whatever links exist — a peer-to-peer distribution scheme decades before BitTorrent.

	Protocol and use
NNTP (Network News Transfer Protocol, RFC 3977; port 119, 563 over TLS)	the text protocol of both reading and server-to-server transfer, with three-digit replies like SMTP
Reading	`LIST` (the groups), `GROUP comp.protocols.tcp-ip` (select; reply: count, first, last article number), `OVER 100-120` (summaries), `ARTICLE n`, `HEAD`, `BODY`, `POST` (followed by the article, ended by a dot)
Server to server	`IHAVE <message-id>` — "do you want it?" (335 send it / 435 not wanted); streaming feeds use `CHECK` and `TAKETHIS` (RFC 4644) to avoid one round trip per article

Usenet shaped Internet culture: Linus Torvalds announced Linux on `comp.os.minix` in 1991, Tim Berners-Lee announced the Web on `alt.hypertext` the same year, and the first commercial spam was posted to thousands of newsgroups in 1994. Discussion has since moved to mailing lists, web forums and social networks — all client–server — but Usenet still runs, and its design remains a classic case of distribution by flooding with duplicate suppression, which we meet again in routing protocols.

7. DNS: services and hierarchy

People use names such as *www.unibuc.ro*; IP packets carry addresses such as 141.85.x.x. The **Domain Name System** (DNS; Paul Mockapetris, 1983; RFCs 1034 and 1035, 1987) is the distributed

database that translates between them. It is itself an application-layer protocol, run by hosts and by name servers, normally over **UDP port 53** — a core Internet function implemented, true to the end-to-end principle, at the edge.

7.1 What DNS provides

- **Name to address**: A (IPv4) and AAAA (IPv6) records.
- **Aliases**: a CNAME record gives the canonical name of an alias (`www.shop.ro` → `shop.cdn-provider.net`).
- **Mail routing**: MX records name the mail servers of a domain (section 3.2).
- **Load distribution**: a busy name can have several addresses, returned in rotating order; CDNs return different addresses to clients in different places.
- **Reverse lookup**: PTR records map an address to a name, under `in-addr.arpa` (IPv4) and `ip6.arpa` (IPv6).
- **Text and service records**: TXT (SPF, DMARC, domain verification), SRV (which host and port offer a service), CAA (which certificate authorities may issue certificates for the domain).

7.2 Why not one big server?

A single server would be a single point of failure, would receive the queries of the whole planet, would be far from most users, and would have to be updated for every name in the world. DNS is instead a **hierarchy of zones**, each managed by its owner:

- the **root** zone, served by 13 named **root server** identities (a.root-servers.net … m.root-servers.net), run by 12 organisations and replicated by anycast at well over a thousand sites; they know the servers of every top-level domain;
- **top-level domains** (TLDs): generic (`.com`, `.org`, `.net`, and hundreds of new ones), country-code (`.ro`, run by ICI Bucharest as RoTLD; `.de`, `.uk`), and `.arpa` for infrastructure; their servers know the servers of each second-level domain;
- **authoritative servers** of each organisation's zone (`unibuc.ro`), which hold its actual records. A zone can **delegate** a subdomain (`fmi.unibuc.ro`) to other servers with NS records.

Two further roles complete the picture. Every host has a **stub resolver** — a library that sends queries. And every network has **recursive resolvers** — run by the ISP or the university, or public ones such as 1.1.1.1, 8.8.8.8 and 9.9.9.9 — which do the work of resolution for the hosts and cache the results.

8. DNS: resolution, caching, records and messages

8.1 Resolving a name

A host asks its local resolver a **recursive** query: "give me the final answer". The resolver, if it has nothing in its cache, asks **iterative** queries: the root server answers with a **referral** to the `.ro` servers; a `.ro` server answers with a referral to the `unibuc.ro` servers, including their addresses (**glue records**) where needed; the authoritative server answers with the record. The resolver returns it to the host and caches every piece.

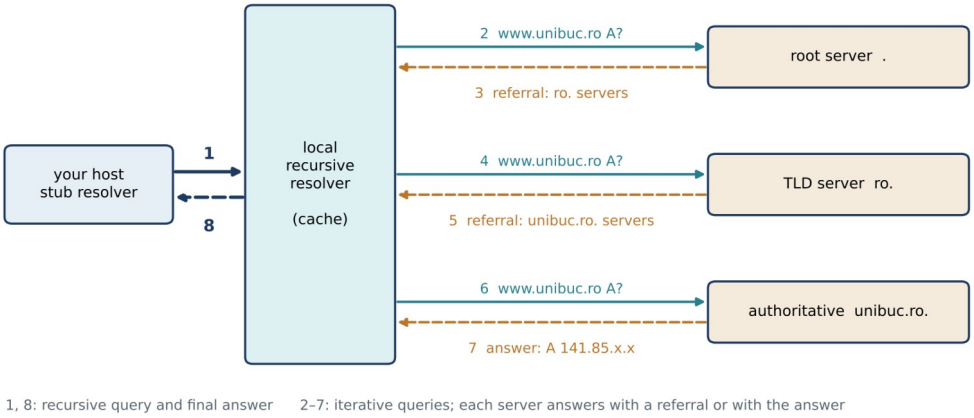


Figure 2. Resolving www.unibuc.ro with a cold cache: one recursive query from the host, three iterative queries from the resolver.

8.2 Caching and TTL

Every record carries a **TTL** (time to live, in seconds), chosen by the zone's owner: how long others may cache it. While it lasts, the resolver answers from its cache, and the TTL it reports counts down. The TLD and root referrals are cached too — for a day or two — so the root servers see only a small fraction of queries. Negative answers ("no such name") are cached as well, for a time given in the zone's SOA record. Caching makes DNS fast and scalable; its price is that changes propagate slowly: an administrator who wants to move a service lowers the TTL a day in advance. Browsers and operating systems keep their own small caches on top.

8.3 Resource records

A zone is a set of **resource records** (RRs), each a tuple *(name, TTL, class, type, value)* — the class is almost always IN, Internet. Part of a zone file:

```
$ORIGIN example.ro.
@      3600  IN SOA   ns1.example.ro. admin.example.ro. (
                     2026102601 7200 900 1209600 300 )
@      86400 IN NS    ns1.example.ro.
@      86400 IN NS    ns2.provider.net.
ns1    86400 IN A     192.0.2.53
@      3600  IN MX    10 mail.example.ro.
mail   3600  IN A     192.0.2.25
www    300   IN A     192.0.2.80
www    300   IN AAAA  2001:db8::80
shop   3600  IN CNAME shops.cdn-provider.net.
@      3600  IN TXT   "v=spf1 mx -all"
```

Type	Value
A / AAAA	an IPv4 / IPv6 address
NS	the name of an authoritative server for the zone (delegation)
CNAME	the canonical name of an alias; the resolver continues with it
MX	a preference and the name of a mail server
SOA	start of authority: primary server, administrator, serial number, refresh and expiry timers, negative-caching TTL
PTR, TXT, SRV, CAA	reverse mapping; free text; service location; allowed certificate authorities

To get a domain, an organisation registers it through a **registrar**; the **registry** of the TLD then publishes in the TLD zone the NS records (and glue) of the organisation's authoritative servers.

8.4 The DNS message

Queries and replies share one format: a 12-byte **header** and four sections. All numbers are in network byte order — you will build these bytes yourself in the laboratory.

Field	Size	Meaning
ID	16 bits	chosen by the client, copied into the reply: matches replies to queries
Flags	16 bits	QR (0 query, 1 reply), opcode, **AA** (authoritative answer), **TC** (truncated), **RD** (recursion desired), **RA** (recursion available), rcode (0 no error, 2 server failure, 3 no such name, 5 refused)
QDCOUNT, ANCOUNT, NSCOUNT, ARCOUNT	4 × 16 bits	the number of entries in the four sections
Question	variable	the name, as labels (`3www6unibuc2ro0`), QTYPE and QCLASS
Answer, Authority, Additional	variable	resource records: the answer; NS records of a referral; extra records such as glue addresses

To keep replies small, a name that already appeared in the message is replaced by a 2-byte **pointer** — its first two bits set, the remaining 14 giving the offset of the earlier copy (`C0 0C` points to the question name at offset 12). A UDP reply was originally limited to 512 bytes; **EDNS(0)** (RFC 6891) lets client and server agree on larger UDP messages, and when an answer still does not fit, the server sets TC and the client repeats the query over **TCP**, which is also used for zone transfers between servers.

8.5 DNS security and privacy

A plain DNS reply is accepted if its ID and port match the query; an attacker who guesses them first can **poison** a resolver's cache, sending all its users to a false address (the Kaminsky attack, 2008; random source ports now make guessing much harder). **DNSSEC** adds digital signatures (RRSIG records, keys in DNSKEY records, a chain of trust through DS records up to the root), so that a resolver can verify an answer's **authenticity**; it does not hide it. **DNS over TLS** (DoT, port 853) and **DNS over HTTPS** (DoH, RFC 8484) encrypt the exchange between host and resolver, so that the café Wi-Fi of Lecture 4 no longer sees which names you look up. Being critical infrastructure, DNS is also a target: in 2016 the Mirai botnet's attack on the DNS provider Dyn made major websites unreachable for hours.

9. NAT: many hosts behind one address

An IPv4 address has 32 bits, about 4.3 billion values, and the regional registries ran out of new blocks between 2011 and 2019. **Network address translation** (RFC 3022) lets a whole network share one public address. Inside, hosts use **private addresses** (RFC 1918), which anyone may use and which are never routed on the Internet: **10.0.0.0/8**, **172.16.0.0/12** and **192.168.0.0/16**.

9.1 How it works

The NAT router rewrites every packet that crosses it. For an outgoing packet it replaces the private source address and port with its own public address and a port it chooses, and records the pair in its **translation table**; for an incoming packet it looks up the destination port in the table and rewrites the destination back to the private address and port. Checksums are updated accordingly. Because ports are translated too, this is properly called NAPT (network address and port translation); in practice everyone says NAT.

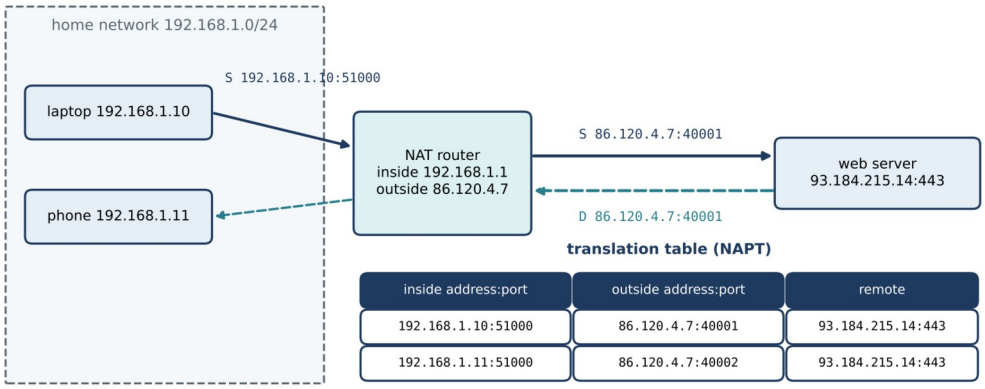


Figure 3. Two hosts with the same source port reach the same server through one public address. The router tells their replies apart by the port it gave each mapping.

9.2 Consequences

- **Capacity.** One public address offers about 64 000 ports per transport protocol for mappings (more, if mappings are reused towards different destinations). ISPs short of addresses run **carrier-grade NAT** (CGNAT), with customers on the shared range 100.64.0.0/10 behind the ISP's own NAT, and allocate each customer a block of ports.
- **Inbound connections are impossible by default.** A packet arriving for a port with no mapping has nowhere to go. A server behind NAT needs **port forwarding** — a static mapping configured on the router — or a mapping requested by the application (UPnP IGD, or PCP, RFC 6887). As a side effect NAT blocks unsolicited traffic, but it is not a substitute for a firewall.
- **Mappings expire.** Idle UDP mappings may vanish after tens of seconds, TCP ones after minutes or hours; long-lived applications send **keepalives**.
- **Addresses inside messages break.** Active FTP (Lecture 4) writes the client's private address in PORT; SIP telephony writes addresses in its messages. Routers include **application-level gateways** that rewrite such messages — impossible when the control connection is encrypted.
- **Peer-to-peer needs help.** Two hosts both behind NAT cannot simply connect. **STUN** (RFC 8489) lets a host learn its public address and port from a server outside; both peers then send packets to each other's public mapping at the same time, opening their NATs (**hole punching**); when that fails, **TURN** (RFC 8656) relays the traffic, and **ICE** (RFC 8445) tries all the options in order — the machinery inside every WebRTC video call.

NAT violates the layering of Lecture 2 — a router rewriting transport headers and even application messages — and the end-to-end principle, since a host's address is no longer the one its peers see. It has nevertheless kept IPv4 alive for three decades. **IPv6**, with 2^{128} addresses, makes it unnecessary: every device can have a globally unique address again, protected, where needed, by a firewall. We return to NAT and IPv6 with the network layer.

Summary

- E-mail is asynchronous: **user agents**, **mail servers** with mailboxes and queues, **SMTP** to push, **POP3/IMAP** to pull, **DNS MX** records to find the receiving server.
- **SMTP** is a text protocol of commands (EHLO, MAIL FROM, RCPT TO, DATA, QUIT) and three-digit replies; the message ends at a line with a single dot (dot-stuffing). The **envelope** (MAIL FROM, RCPT TO) is distinct from the **header** (From:, To:). Ports 25 (relay), 587 and 465 (submission).
- A message is a **header** and a **body** (RFC 5322). **MIME** adds content types, multipart bodies and transfer encodings; **base64** grows binary data by about 37 %.
- **POP3** downloads mail from a mailbox (states AUTHORIZATION, TRANSACTION, UPDATE). **IMAP** keeps mail, folders and flags on the server, with tagged commands, server-side search and partial fetch. TLS protects each hop; **S/MIME** or **OpenPGP** protect the message end to end; **SPF**, **DKIM** and **DMARC** fight forgery.
- **Usenet** distributes articles in newsgroups among news servers by **flooding** with duplicate suppression by Message-ID and loop avoidance by the Path field; **NNTP** serves both readers and server-to-server feeds.
- **DNS** is a hierarchical, distributed, cached database: root, TLD and authoritative servers; recursive resolvers performing iterative queries; **TTL**-based caching; resource records (A, AAAA, NS, CNAME, MX, SOA, PTR, TXT); messages with a 12-byte header and four sections, name compression, UDP port 53 with TCP fallback; DNSSEC for authenticity, DoT/DoH for privacy.
- **NAT** maps many private addresses (RFC 1918) to one public address by rewriting addresses and ports and keeping a translation table. It blocks inbound connections unless ports are forwarded, breaks protocols that carry addresses, and forces peer-to-peer applications to use STUN, TURN and ICE. IPv6 removes the need for it.

Key terms

user agent · mail server · mailbox · message queue · submission server · mail transfer agent · SMTP · EHLO · MAIL FROM · RCPT TO · DATA · dot-stuffing · envelope · header · reverse path · bounce · port 25 / 587 / 465 · STARTTLS · MX record · RFC 5322 · Received · Message-ID · MIME · Content-Type · multipart · boundary · Content-Transfer-Encoding · base64 · quoted-printable · encoded word · POP3 · AUTHORIZATION / TRANSACTION / UPDATE · UIDL · IMAP · folder · flag · tag · untagged response · IDLE · webmail · JMAP · S/MIME · OpenPGP · SPF · DKIM · DMARC · Usenet · newsgroup · article · hierarchy · flooding · Path · NNTP · IHAVE · DNS · stub resolver · recursive resolver · root server · TLD · authoritative server · zone · delegation · glue record · recursive / iterative query · referral · TTL · negative caching · resource record · A · AAAA · NS · CNAME · MX · SOA · PTR · TXT · SRV · CAA · registrar · registry · DNS header · QR · AA · TC · RD · RA · rcode · name compression · EDNS(0) · cache poisoning · DNSSEC · DoT · DoH · NAT · NAPT · private address · RFC 1918 · translation table · CGNAT · port forwarding · UPnP · PCP · keepalive · application-level gateway · STUN · hole punching · TURN · ICE · IPv6

Review questions

1. Why does a mail message usually pass through two mail servers instead of going directly from the sender's computer to the reader's?

2. Which SMTP commands form the envelope? Give two situations in which the envelope recipient does not appear in the `To:` header.
3. How does SMTP know where a message ends? What happens to a line of the message that starts with a dot?
4. Why must binary attachments be encoded, and by how much does base64 increase their size?
5. A user reads mail on a phone, a laptop and in a browser. Why is IMAP better than POP3 for her?
6. What does STARTTLS protect, and what does it not protect?
7. How does a Usenet server avoid storing an article twice, and avoid sending it back to where it came from?
8. Why is DNS distributed instead of centralised? Give three reasons.
9. What is the difference between a recursive and an iterative query? Which servers usually answer which?
10. Why might a website that changed its address still be unreachable for some users for hours?
11. What does the TC bit in a DNS reply mean, and what does the client do?
12. Why can a home user not run a web server reachable from the Internet without configuring the router? What must be configured?

Exercises

1. A message contains 2 KB of text and a 6 MB PDF attachment, encoded in base64 with lines of 76 characters plus CR LF. Estimate the size of the message on the wire (1 MB = 10^6 bytes).
2. A host's RTT to its local resolver is 3 ms; the resolver's RTTs to a root server, a TLD server and an authoritative server are 25 ms, 35 ms and 20 ms. Ignore processing. How long does the host wait for the address of www.example.ro (a) with an empty cache; (b) when the resolver has cached the .ro referral; (c) when it has cached the answer?
3. Write the bytes of the question section of a DNS query for the AAAA record of `ftp.unibuc.ro`.
4. A NAT router with public address 86.120.4.7 receives, from 192.168.1.20:50000 to 1.1.1.1:53 (UDP), and then from 192.168.1.21:50000 to 1.1.1.1:53. Write the translated packets and the translation table, assuming the router allocates ports from 61000.
5. An ISP with 40 000 customers runs CGNAT and gives each customer a block of 2 000 ports per public address. How many public IPv4 addresses does it need (use 64 000 usable ports per address)?

Answers

1. 6 000 000 bytes → 8 000 000 base64 characters; 8 000 000 / 76 ≈ 105 264 lines, adding 2 bytes each ≈ 210 528 bytes; plus about 2 KB of text and the headers: **≈ 8.2 MB** — 37 % more than the attachment.
2. (a) 3 + 25 + 35 + 20 = **83 ms**. (b) 3 + 20 = **23 ms**. (c) **3 ms** — and 0 ms if the host's own cache has it.
3. `03 66 74 70 06 75 6e 69 62 75 63 02 72 6f 00` (the labels ftp, unibuc, ro and the root), then QTYPE `00 1c` (28 = AAAA) and QCLASS `00 01` (IN): 19 bytes.

4. Packet 1 becomes 86.120.4.7:61000 → 1.1.1.1:53; packet 2 becomes 86.120.4.7:61001 → 1.1.1.1:53. Table: 192.168.1.20:50000 ↔ 86.120.4.7:61000 and 192.168.1.21:50000 ↔ 86.120.4.7:61001 (UDP, remote 1.1.1.1:53). Replies to 61000 and 61001 are translated back to the right host, although both hosts used the same source port.
5. 64 000 / 2 000 = 32 customers per address; 40 000 / 32 = **1 250 public addresses** instead of 40 000.

Reading and next steps

Required

- Kurose and Ross, *Computer Networking: A Top-Down Approach*, 8th edition, Chapter 2: electronic mail in the Internet; DNS — the Internet's directory service. Chapter 4: the section on NAT.

Recommended

- RFC 5321 (SMTP), sections 2–4; RFC 5322 (message format); RFC 2045 (MIME); RFC 1939 (POP3, short and readable); RFC 9051 (IMAP4rev2), section 2.
- RFC 3977 (NNTP), sections 3–6; RFC 1034 (DNS concepts), sections 2–5; RFC 1035, section 4 (message format).
- RFC 3022 (traditional NAT); RFC 4787 (NAT behaviour for UDP); RFC 8445 (ICE), section 2.

Laboratory 5 (this week)

You send mail by typing SMTP and with a MIME attachment, read it with POP3, look inside an IMAP greeting, follow DNS resolution with dig, build DNS queries byte by byte over UDP sockets, and build a NAT router of your own from Linux network namespaces — or, on Windows, find the NATs between you and the Internet.

Next lecture

The transport layer I: multiplexing and demultiplexing, UDP, and the principles of reliable data transfer.

PART B · WEEK 5

Seminar 5: messages, names and translated addresses

Two-hour problem session (2 × 50 minutes) accompanying Lecture 5

The seminar turns Lecture 5 into careful reading of protocol dialogues and bytes, small calculations and designs. Work in pairs; one pair presents each problem.

By the end of the seminar you should be able to

- read an SMTP dialogue and separate the envelope from the header;
- estimate the size of a MIME message and read the path of a message from its Received fields;
- choose between POP3 and IMAP and read an IMAP exchange;
- follow the flooding of an article between Usenet servers;
- compute DNS resolution times with and without caching, decode a DNS reply byte by byte and write a small zone;
- fill in a NAT translation table and reason about NAT capacity, port forwarding and peer-to-peer traffic.

Plan

Step	Activity	Time
A	Problems 1–2: an SMTP dialogue; the size and path of a message	17 min
B	Problems 3–4: POP3 or IMAP; reading IMAP	13 min
C	Problem 5: flooding news	10 min
	Break	10 min
D	Problems 6–8: DNS — timing, bytes and a zone	25 min
E	Problems 9–11: NAT tables, capacity and traversal	25 min
	Homework	(in E)

A. Electronic mail on the wire

Problem 1

A server logged the following session (C client, S server):

```
S: 220 mx.example.com ESMTP
C: EHLO mail.uni.ro
S: 250-mx.example.com
S: 250-SIZE 26214400
S: 250 STARTTLS
C: MAIL FROM:<list-bounces@uni.ro>
S: 250 OK
C: RCPT TO:<bob@example.com>
S: 250 OK
C: RCPT TO:<carol@example.com>
S: 550 5.1.1 No such user
C: RCPT TO:<dan@example.com>
```

```
S: 250 OK
C: DATA
S: 354 Go ahead
C: From: Prof. Ionescu <ionescu@uni.ro>
C: To: Students <cn-2026@uni.ro>
C: Subject: Exam room
C:
C: The exam is in room 105.
C: ..and bring your student card.
C: .
S: 250 OK queued as 91BC
C: QUIT
S: 221 Bye
```

1. Who receives the message on this server? What will the sender learn about carol, and from whom?
2. None of the recipients appears in the `To:` field. Explain why the message is still delivered. Name the two addresses of the envelope and the two of the header.
3. What is the last line of the body as Bob reads it?
4. Which important step is missing between EHLO and MAIL FROM, given the server's extensions? What could an observer on the path read?

Solution

1. Bob and Dan (both RCPT answered 250). Carol was refused with a permanent error (550) during the session, so the *sending* server — mail.uni.ro — generates a bounce to the reverse path `list-bounces@uni.ro`, i.e. to the list software, not to the professor.

2. Delivery uses only the envelope: `MAIL FROM:<list-bounces@uni.ro>` and the `RCPT TO` addresses. The header `From: ionescu@uni.ro` and `To: cn-2026@uni.ro` are only displayed. This is how mailing lists (and Bcc) work.

3. `..and bring your student card.` — the line started with a dot, so the client doubled it (dot-stuffing) and the server removes one dot.

4. STARTTLS (then EHLO again inside TLS). Without it, anyone on the path reads the whole dialogue: addresses, subject and body.

Problem 2

Bob receives a message with a 4.5 MB photograph attached (base64, 76 characters per line plus CR LF) and 1 KB of text. Its header begins:

```
Received: from mx1.example.com (mx1.example.com [198.51.100.7])
    by imap.example.com with LMTP; Mon, 26 Oct 2026 09:14:05 +0200
Received: from mail.uni.ro (mail.uni.ro [193.231.20.10])
    by mx1.example.com with ESMTPS; Mon, 26 Oct 2026 09:14:03 +0200
Received: from [192.168.1.34] (unknown [86.120.4.7])
    by mail.uni.ro with ESMTPSA; Mon, 26 Oct 2026 09:13:58 +0200
From: Alice <alice@uni.ro>
```

1. Estimate the size of the message (1 MB = 10^6 bytes). Will the server of Problem 1 accept it?
2. List the hops of the message in order, from Alice's computer to Bob's mailbox. Which hop took longest?
3. What do ESMTPSA and the two addresses in the last `Received` field tell you about Alice?

Solution

1. 4 500 000 × 4/3 = 6 000 000 characters; 6 000 000 / 76 ≈ 78 948 lines × 2 bytes ≈ 158 kB of line ends; total ≈ **6.16 MB** plus a few kB of text and header. The SIZE limit of 26 214 400 bytes (25 MiB) is not exceeded.
2. Read from the bottom up: Alice's computer → mail.uni.ro (09:13:58) → mx1.example.com (09:14:03) → imap.example.com, Bob's mailbox server (09:14:05). The relay hop from mail.uni.ro to mx1.example.com took longest (5 s, including queueing and spam checks); the clocks of different servers may differ slightly, so such times are only indicative.
3. ESMTP with **S**ecurity (TLS) and **A**uthentication: Alice submitted the message after logging in. Her computer called itself 192.168.1.34 — a private address — but the connection came from 86.120.4.7: she is behind a NAT router (section 9 of the lecture).

B. Reading mail

Problem 3

For each user, say whether POP3 or IMAP is more suitable and why:

1. a researcher on a ship, with a satellite link for one hour a day, who reads mail only on one laptop;
2. a manager who reads mail on a phone, a tablet and an office PC, and files messages in folders;
3. a shop's order system that collects order confirmations from a mailbox every minute and processes each once;
4. a student who wants to read the subjects of 3 000 messages over a slow mobile link, then only two of the messages.

Solution

1. **POP3**: download everything during the link window, read and keep it off line; nothing needs to stay synchronised with other devices.
2. **IMAP**: folders, read flags and deletions are kept on the server and seen identically from all three devices.
3. **POP3** is enough (or IMAP): UIDL gives each message a stable identifier so that the program fetches each message once, then deletes it with `DELE`.
4. **IMAP**: `FETCH 1:3000 (BODY.PEEK[HEADER.FIELDS (SUBJECT)])` transfers only subjects; the two messages are then fetched in full. With POP3, `TOP n 0` per message is possible, but takes 3 000 commands and gives no server-side search or folders.

Problem 4

```
C: t1 SELECT INBOX
S: * 12 EXISTS
S: * OK [UIDNEXT 4392] Predicted next UID
S: t1 OK [READ-WRITE] SELECT completed
C: t2 SEARCH UNSEEN FROM "ionescu"
S: * SEARCH 9 12
S: t2 OK SEARCH completed
C: t3 FETCH 12 (RFC822.SIZE BODY.PEEK[HEADER.FIELDS (SUBJECT)])
S: * 12 FETCH (RFC822.SIZE 6158210 BODY[HEADER.FIELDS (SUBJECT)] {22}
S: Subject: Exam room
S: )
S: t3 OK FETCH completed
C: t4 STORE 9 +FLAGS (\Deleted)
C: t5 EXPUNGE
```

1. How many messages are in the folder, and which unread messages come from Ionescu? Where was this search done?
2. Why `BODY.PEEK` and not `BODY`? What does `{22}` announce?
3. Message 12 is large. What would a careful mobile client do next?
4. What happens to message 9 after t4? After t5?

> **Solution**
> 1. 12 messages; 9 and 12 are unread and from Ionescu. The search ran **on the server**, so no message crossed the network.
> 2. `BODY.PEEK` does not set the `\Seen` flag, so the message stays unread on all devices. `{22}` is a length prefix: 22 bytes of literal data follow (the subject line and CR LF, blank line) before the closing parenthesis.
> 3. Not download 6 MB: fetch the structure (`BODYSTRUCTURE`), show the text part only (`BODY[1]`), and fetch the attachment when the user asks for it.
> 4. After t4 it is only marked `\Deleted` (still present, still visible to other clients); `EXPUNGE` removes it permanently — the IMAP counterpart of POP3's UPDATE state.

C. Flooding news

Problem 5

Five news servers are connected as follows: A–B, A–C, B–C, B–D, C–D, D–E. An article is posted on A at time 0. Every server offers each new article to all its neighbours except those already named in the article's `Path:`; an offer and transfer over one link take 1 minute. A server accepts an article only if it has not seen its Message-ID.

1. At what time does each server first receive the article? Write the `Path:` as stored on E.
2. How many transfers are offered in total, and how many are refused as duplicates?
3. Why does the Path field alone not prevent duplicates, and why is the Message-ID check alone enough to stop the flood?

> **Solution**
> 1. A: 0. B and C: 1 (from A). D: 2 (from B or C — both offer it during the second minute; say B is first). E: 3 (from D). Each server prepends its name when it accepts the article, so E stores `Path: E!D!B!A!not-for-mail`.
> 2. A offers to B and C (2 accepted). B (Path B!A) offers to C and D; C (Path C!A) offers to B and D: B→C and C→B are refused, B→D accepted, C→D refused. D (Path D!B!A) offers to C (refused) and E (accepted). E has no other neighbour. Total **8 offers: 4 accepted (one per server other than A), 4 refused**. With IHAVE, a refusal costs only a short exchange, not the article.
> 3. Path only lists the servers on *this copy's* route; B's copy and C's copy follow different routes and neither names the other server. The Message-ID check makes each server accept the article once, so each server floods it once: the total work is bounded by the number of links.

D. DNS

Problem 6

A laptop's RTT to the faculty resolver is 2 ms. The resolver's RTTs: root 20 ms, `.ro` servers 30 ms, `unibuc.ro` servers 15 ms, `.com` servers 25 ms, `cdn-provider.com` servers 40 ms. Records:

`www.unibuc.ro` is a CNAME for `unibuc.cdn-provider.com` (TTL 3600), which has an A record with TTL 60. TLD referrals are cached 2 days. Ignore processing and TCP.

1. The resolver has just started (empty cache). How long until the laptop has the address of www.unibuc.ro?
2. A second laptop asks for the same name 30 s later; a third, 5 minutes later. How long does each wait?
3. Why might the CDN choose such a short TTL for the A record?

> **Solution**
>
> 1. Root (20) → `.ro` server (30) → `unibuc.ro` server (15), which returns the CNAME. The resolver must now resolve the canonical name under `.com`: the root again (20, since only the `.ro` referral is cached), then a `.com` server (25), then a `cdn-provider.com` server (40). Total = 2 + 20 + 30 + 15 + 20 + 25 + 40 = **152 ms**. (Real resolvers usually have the root zone and popular TLDs cached long before.)
> 2. After 30 s everything is cached: **2 ms**. After 5 minutes the CNAME (TTL 3600) and the `.com` referral are cached but the A record (TTL 60) has expired: one query to `cdn-provider.com`: 2 + 40 = **42 ms**.
> 3. So that it can change its answer quickly — to steer clients to a nearer or less loaded server, or away from a failed one. The price is more queries to its servers and a slower first visit.

Problem 7

A reply captured on UDP port 53 (offsets in hexadecimal on the left):

```
0000  1a 2b 81 80 00 01 00 02 00 00 00 00 03 77 77 77
0010  07 65 78 61 6d 70 6c 65 02 72 6f 00 00 01 00 01
0020  c0 0c 00 05 00 01 00 00 01 2c 00 06 03 77 65 62
0030  c0 10 c0 2c 00 01 00 01 00 00 01 2c 00 04 c0 00
0040  02 50
```

1. Decode the header: ID, every flag, the four counts.
2. Decode the question.
3. Decode the two answers, resolving every compression pointer. What is the address, and under which names?
4. How many bytes did compression save?

Solution

1. ID 0x1a2b. Flags 0x8180 = 1000 0001 1000 0000: QR = 1 (reply), opcode 0, AA = 0 (not authoritative — it came from a resolver's cache or recursion), TC = 0, RD = 1 (copied from the query), RA = 1, rcode 0 (no error). QDCOUNT 1, ANCOUNT 2, NSCOUNT 0, ARCOUNT 0.
2. From offset 0x0c: `03 www 07 example 02 ro 00` = www.example.ro; QTYPE 0x0001 = A, QCLASS 0x0001 = IN.
3. Answer 1 at 0x20: name `c0 0c` → offset 12 = www.example.ro; type 5 = **CNAME**, class IN, TTL 0x12c = 300 s, RDLENGTH 6; data `03 web c0 10` = web + pointer to offset 0x10 (example.ro) = **web.example.ro**. Answer 2 at 0x32: name `c0 2c` → offset 0x2c, where `03 web` begins = web.example.ro; type 1 = **A**, class IN, TTL 300, RDLENGTH 4, data `c0 00 02 50` = **192.0.2.80**. So www.example.ro is an alias of web.example.ro, whose address is 192.0.2.80.
4. Without pointers: answer 1's name would be 16 bytes instead of 2 (14 saved), its data 16 instead of 6 (10 saved — `web` + `example.ro` written out), answer 2's name 16 instead of 2 (14 saved): **38 bytes** saved, a third of the 104-byte uncompressed message.

Problem 8

You run the domain `retele-lab.ro`. You have two name servers (193.0.2.1 and one at a hosting provider, `ns.host.net`), a web server at 193.0.2.80 that should also answer to `retele-lab.ro` and `www.retele-lab.ro`, a mail server at 193.0.2.25 with a backup at the provider (`mx.host.net`), and only your mail server may send mail for the domain.

1. Write the zone's records (TTLs of your choice; omit the SOA).
2. What must the `.ro` registry publish, and why is a glue record needed for one server but not the other?
3. You will move the web server to a new address next Friday. What do you do on Thursday?

Solution

1. One possible zone:
`@ 86400 IN NS ns1.retele-lab.ro.` · `@ 86400 IN NS ns.host.net.` · `ns1 86400 IN A 193.0.2.1` · `@ 3600 IN A 193.0.2.80` · `www 3600 IN CNAME retele-lab.ro.` · `@ 3600 IN MX 10 mail.retele-lab.ro.` · `@ 3600 IN MX 20 mx.host.net.` · `mail 3600 IN A 193.0.2.25` · `@ 3600 IN TXT "v=spf1 ip4:193.0.2.25 -all"`. (The apex `@` cannot be a CNAME, because it already has NS and MX records; `www` can.)
2. The NS records of `retele-lab.ro` (ns1.retele-lab.ro and ns.host.net) in the `.ro` zone, and the A record of `ns1.retele-lab.ro` as **glue**: a resolver could not otherwise find the address of ns1 — it is inside the zone it serves. `ns.host.net` is resolved through `.net`, so it needs no glue.
3. Lower the TTL of the A record (e.g. to 60 s) at least one old TTL (1 hour) before the move, so that caches hold the old address for at most a minute after Friday's change; raise it again afterwards.

E. NAT

Problem 9

A home router with public address 86.120.4.7 and LAN 192.168.1.0/24 uses port-preserving NAT: it keeps the source port when that port is free on its public side, otherwise it chooses the next free port from 40000. In order:

1. 192.168.1.10:51000 → 142.250.1.1:443 (TCP)
2. 192.168.1.11:51000 → 142.250.1.1:443 (TCP)
3. 192.168.1.11:5353 → 1.1.1.1:53 (UDP)
4. 142.250.1.1:443 → 86.120.4.7:40000 (TCP)
5. 203.0.113.9:22 → 86.120.4.7:22 (TCP SYN)

Fill in the translation table and give each packet after translation. Then the router is configured to forward TCP port 2222 to 192.168.1.10:22: what does a connection from 203.0.113.9:60000 to 86.120.4.7:2222 look like on both sides?

> **Solution**
> 1. 86.120.4.7:51000 → 142.250.1.1:443. Entry: TCP 192.168.1.10:51000 ↔ 86.120.4.7:51000.
> 2. Port 51000 is taken (towards the same destination): 86.120.4.7:40000 → 142.250.1.1:443. Entry: TCP 192.168.1.11:51000 ↔ 86.120.4.7:40000.
> 3. UDP is a separate table: 86.120.4.7:5353 → 1.1.1.1:53. Entry: UDP 192.168.1.11:5353 ↔ 86.120.4.7:5353.
> 4. Matches entry 2: delivered as 142.250.1.1:443 → 192.168.1.11:51000.
> 5. No entry for port 22: the router drops the packet (or answers with a TCP RST): inbound connections are impossible without port forwarding.
> With forwarding: outside, 203.0.113.9:60000 → 86.120.4.7:2222; inside, 203.0.113.9:60000 → 192.168.1.10:22. The SSH server sees the real client address; replies are translated back from 192.168.1.10:22 to 86.120.4.7:2222.

Problem 10

1. An ISP has 2 048 public IPv4 addresses and 100 000 customers behind CGNAT. Using 64 000 usable ports per address, how many ports can each customer have? A browser opens about 50 TCP connections for a heavy page, and an online game holds 20 UDP flows. Is the allocation sufficient for a family of four?
2. A messaging app keeps an idle TCP connection open to receive notifications. The CGNAT removes idle TCP mappings after 10 minutes, a home router after 2 hours. How often must the app send a keepalive, and what happens otherwise?
3. Why does a CGNAT customer have no way to forward a port at all?

Solution

1. 2 048 × 64 000 / 100 000 ≈ **1 310 ports per customer**. Four people with a few browsers (each tab possibly 20–50 connections), phones, games and updates may reach several hundred simultaneous mappings: sufficient in normal use, though heavy users can run out, which shows as pages that load partly. ISPs allocate blocks dynamically or use larger blocks for this reason.
2. More often than the shortest timeout on the path — e.g. every 5–9 minutes. Otherwise the CGNAT drops the mapping silently; the server's next notification arrives for a port with no mapping and is lost, and the app discovers this only when it next sends.
3. The mapping is created on the ISP's NAT, which the customer does not control: the home router's own port forwarding only reaches the customer's private side. The customer needs a public IPv4 address from the ISP (often paid), IPv6, or a relay or tunnel service.

Problem 11

Alice and Bob, each behind a home NAT, want a video call. Their computers know only their private addresses.

1. Why can neither simply send UDP packets to the other's private address? To the other's public address?
2. Describe how STUN and hole punching let them exchange packets directly. What property must the NATs have?
3. When does the call have to go through TURN, and what does that cost?

Solution

1. Private addresses are not routed on the Internet. A packet to the other's *public* address arrives at a NAT with no mapping for it and is dropped.
2. Each sends a STUN Binding request to a server and learns its public address and port (its mapping). They exchange these through the signalling server (the app's server, over its existing connection). Then both send UDP packets to each other's public mapping at about the same time: Alice's outgoing packet creates in her NAT a mapping that now accepts Bob's packets, and vice versa. It works if the NATs are **endpoint-independent**: they use the same public port for the same private socket whatever the destination (RFC 4787), so the address learned from STUN is the one the peer must use.
3. When a NAT gives a different public port for each destination (symmetric NAT) or a firewall blocks UDP: a TURN server outside relays all the media. The cost: higher delay and the operator pays for the relay's bandwidth. ICE tries the direct candidates first and falls back to TURN.

Homework

1. Find a message in your own mailbox (in Gmail: ⋮ → *Show original*). Copy its `Received` fields and the results of SPF, DKIM and DMARC (`Authentication-Results`). Draw the message's path and say which server checked what.
2. With `dig +norecurse`, resolve `www.unibuc.ro` by hand, starting at `a.root-servers.net` and following each referral. Show each command and the section of the reply that sent you on (Authority / Additional).
3. On a sheet of paper, write the full DNS query (header and question, in hexadecimal) for the MX record of `unibuc.ro` with ID 0x0042 and recursion desired. Check it against your C client in Laboratory 5 with tcpdump.

4. Explain in half a page why a NAT router must rewrite the PORT command of active FTP but cannot do so for FTPS, and what Laboratory 4's passive mode changes.

PART C · WEEK 5

Laboratory 5 (Linux): mail by hand, DNS byte by byte, a NAT router of your own

Two-hour laboratory sheet (2 × 50 minutes) accompanying Lecture 5

This week you speak SMTP and POP3 yourself to small servers on your own machine, look at a real IMAP server's greeting, follow DNS with dig and with a DNS client written in C over UDP sockets, and build a home network with a NAT router inside one Linux machine with network namespaces. All programs were tested on Ubuntu 24.04 and are provided as files.

By the end of the laboratory you should be able to

- send a message by typing SMTP, and see dot-stuffing and the envelope at work;
- send a MIME message with an attachment and read its encoded form;
- read and delete mail with POP3 by hand, and open an IMAP session over TLS;
- follow DNS resolution, record types and TTLs with dig, and query DNS over HTTPS;
- build a DNS query in C, send it over UDP and decode the reply, including compressed names;
- build a NAT router, watch its translation table, configure port forwarding, and find your own public mapping with STUN.

Before you start

- **Environment.** The course VM or any Linux where you have sudo (Task 6 creates network namespaces). Install the tools once (on Ubuntu `python3-aiosmtpd` is in the universe repository; elsewhere `pip install aiosmtpd`):

```
sudo apt install -y build-essential dnsutils tcpdump curl netcat-openbsd \
    nftables conntrack python3-aiosmtpd
```

- **Files.** Unpack the archive of this laboratory into `lab5/`: `linux/` (dns_query.c, Makefile, nat_lab.sh), `python/` (talk.py, send_mail.py, pop3_server.py, dns_query.py, stun_client.py) and `windows/`. Build with `make -C linux`.
- **talk.py** is a small terminal for text protocols: each line you type is sent with CR LF, each line received is printed after `<-`. `nc -C host port` does the same without TLS.
- **Ethics.** The mail servers in Tasks 1–3 are your own. Never send mail by hand through someone else's server, and never try passwords on accounts that are not yours.
- **What to hand in.** A short report with answers to **Q1–Q16** and the outputs that support them; your extended `dns_query.c`.

Plan

Step	Activity	Time
1	SMTP by hand: a mail server of your own	15 min
2	MIME: an attachment on the wire	10 min
3	Reading mail: POP3 by hand, an IMAP greeting	15 min
	Break	10 min

Step	Activity	Time
4	DNS with dig: resolution, records, TTL, DoH	15 min
5	A DNS client in C over UDP	20 min
6	A NAT router of your own; STUN	15 min

Task 1 — SMTP by hand

`aiosmtpd` is a small SMTP server written in Python. With the `Mailbox` handler it stores every accepted message as one file in a **Maildir** — the mailbox format in which each message is a file in `new/`.

```
cd lab5
# terminal A: an SMTP server on port 8025, storing mail in ~/Maildir
python3 -m aiosmtpd -n -l 127.0.0.1:8025 -c aiosmtpd.handlers.Mailbox ~/Maildir
# terminal B: speak SMTP
python3 python/talk.py 127.0.0.1 8025          # or: nc -C 127.0.0.1 8025
```

Type the following lines one by one, waiting for each reply (the lines starting with `<-` are the server's):

```
EHLO client.lab.test
MAIL FROM:<alice@lab.test>
RCPT TO:<bob@lab.test>
RCPT TO:<carol@lab.test>
DATA
From: Alice <alice@lab.test>
To: Bob <bob@lab.test>
Subject: typed by hand

Hello Bob,
..this line started with one dot.
.
QUIT

ls ~/Maildir/new/                    # one file per message
cat ~/Maildir/new/*                  # header added by the server, then yours
```

- **Q1.** Copy the server's reply to EHLO. Which extensions does it announce? Which reply codes did you get for MAIL, RCPT, DATA, the final dot and QUIT, and what class (2xx/3xx/4xx/5xx) is each?
- **Q2.** Carol is not named in any header line. Does she receive the message? Which header fields did the server add, and where does the envelope appear in the stored file (`X-Peer`, `X-MailFrom`, `X-RcptTo`)?
- **Q3.** How does the line you typed with two dots appear in the stored message? Why was the second dot needed? Start a new message with a body line `.` followed by more text: what happens?

Task 2 — MIME: an attachment on the wire

```
"""send_mail.py - send a MIME message with an attachment over SMTP (Lab 5).
usage: python3 send_mail.py [server] [port] [attachment]
       (default server 127.0.0.1, port 8025)"""
import mimetypes, smtplib, sys
from email.message import EmailMessage
from pathlib import Path
```

```
server = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
port = int(sys.argv[2]) if len(sys.argv) > 2 else 8025

msg = EmailMessage()                          # the RFC 5322 header fields
msg["From"] = "Alice Student <alice@lab.test>"
msg["To"] = "Bob Teacher <bob@lab.test>"
msg["Subject"] = "Lab 5 report — first draft"     # non-ASCII: RFC 2047
msg.set_content("Hello Bob,\n\nthe report is attached.\n\nAlice\n")
if len(sys.argv) > 3:                 # MIME: the body becomes multipart
    path = Path(sys.argv[3])
    ctype = mimetypes.guess_type(path.name)[0] or "application/octet-stream"
    maintype, subtype = ctype.split("/", 1)
    msg.add_attachment(path.read_bytes(), maintype=maintype,
                       subtype=subtype, filename=path.name)

with smtplib.SMTP(server, port) as smtp:
    smtp.set_debuglevel(1)                    # show the SMTP dialogue
    smtp.ehlo("client.lab.test")
    # the envelope (MAIL FROM, RCPT TO) is separate from From:/To:
    smtp.send_message(msg, from_addr="alice@lab.test",
                      to_addrs=["bob@lab.test"])
```

```
head -c 30000 /dev/urandom > photo.bin          # 30 000 random bytes
python3 python/send_mail.py 127.0.0.1 8025 photo.bin
ls -l photo.bin ~/Maildir/new/
less "$(ls -t ~/Maildir/new/* | head -1)"       # the newest message
```

- **Q4.** In the debug output, identify the envelope (`MAIL FROM`, `RCPT TO`) and the start of the message after DATA. How was the non-ASCII subject (the dash —) written in the header? Decode it by hand or with `python3 -c "import base64; print(base64.b64decode('…'))"`.
- **Q5.** Find the `Content-Type` of the whole message and of each part, the boundary and the `Content-Transfer-Encoding` of the attachment. Compare the sizes: how much larger is the stored message than `photo.bin`, as a percentage? Does it match the lecture's 37 %?

Task 3 — Reading mail: POP3 and IMAP

`pop3_server.py` serves the messages in `~/Maildir/new` to user `bob`, password `secret`. It implements the three states of the lecture and deletes nothing until `QUIT`.

```
# terminal C
python3 python/pop3_server.py ~/Maildir 1110
# terminal B
python3 python/talk.py 127.0.0.1 1110
```

```
USER bob
PASS secret
STAT
LIST
UIDL
TOP 1 0
RETR 1
DELE 1
RSET
DELE 1
QUIT
```

Now look at a real IMAP server over TLS. You only ask for its capabilities and log out — no account is needed:

```
python3 python/talk.py imap.gmail.com 993 --tls
a1 CAPABILITY
a2 LOGOUT
openssl s_client -connect imap.gmail.com:993 -quiet     # the same, with openssl
```

- **Q6.** Give the POP3 reply to STAT and LIST and explain the numbers. How does RETR mark the end of the message? What would happen to message 1 if you closed the connection without QUIT? Check with `ls ~/Maildir/new` after a session ending in `DELE 1` and QUIT.
- **Q7.** Copy the IMAP greeting and the CAPABILITY line. Find `IMAP4rev1`, `IDLE` and the authentication methods (AUTH=…). Which lines are tagged and which untagged? Why can you not see this dialogue with tcpdump?

Task 4 — DNS with dig

```
dig www.unibuc.ro                          # the four sections; the resolver used
dig +short www.unibuc.ro A; dig +short unibuc.ro MX; dig +short unibuc.ro NS
dig +trace www.unibuc.ro                   # iterative resolution from the root
dig +norecurse @a.root-servers.net www.unibuc.ro   # one referral
dig @1.1.1.1 www.google.com A              # twice, a few seconds apart: TTL
dig gmail.com TXT; dig _dmarc.gmail.com TXT        # SPF and DMARC
dig -x 8.8.8.8                             # reverse lookup: PTR
dig +dnssec unibuc.ro SOA                  # RRSIG records, if signed
curl -s -H 'accept: application/dns-json' \
    'https://cloudflare-dns.com/dns-query?name=unibuc.ro&type=MX'
```

- **Q8.** In the first `dig`, read the header line: which flags are set (qr, rd, ra, aa)? Which server answered and how long did it take? Repeat the command: why is the time shorter now?
- **Q9.** In the `+trace` output, name the servers that answered at each level and the record types that sent the resolver on. In the `+norecurse` query to the root, in which section are the `.ro` servers, and why is there an Additional section?
- **Q10.** How did the TTL of www.google.com change between the two queries to 1.1.1.1? What do the SPF and DMARC records of gmail.com say? Why does the DoH query not appear in `tcpdump port 53`?

Task 5 — A DNS client in C

The program builds a query exactly as in section 8.4 of the lecture — the 12-byte header in network byte order, the name as labels, QTYPE and QCLASS — sends it in one UDP datagram, waits up to 2 seconds for the reply (three tries), and decodes every resource record, following compression pointers in `read_name`.

```
/* dns_query.c - build a DNS query by hand and send it over UDP
   (Computer Networks, Lab 5)
   usage: ./dns_query name [type] [server] [port]
   e.g.   ./dns_query www.kernel.org AAAA 1.1.1.1                       */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <time.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/time.h>

static const struct { const char *name; int code; } types[] = {
    {"A", 1}, {"NS", 2}, {"CNAME", 5}, {"SOA", 6},
    {"PTR", 12}, {"MX", 15}, {"TXT", 16}, {"AAAA", 28} };
```

```
#define NTYPES (int)(sizeof types / sizeof types[0])

static const char *type_name(int code) {
    for (int i = 0; i < NTYPES; i++)
        if (types[i].code == code) return types[i].name;
    return "?";
}

/* Read a 16-bit number stored in network byte order. */
static uint16_t get16(const unsigned char *p) {
    return (uint16_t)(p[0] << 8 | p[1]);
}

/* Read a (possibly compressed) name at msg[*pos] into out and move *pos
   past it. A byte 11xxxxxx starts a 2-byte pointer to an earlier name. */
static int read_name(const unsigned char *msg, int len, int *pos,
                     char *out, int cap) {
    int p = *pos, jumped = 0, n = 0, hops = 0;
    for (;;) {
        if (p >= len || hops++ > 50) return -1;
        int l = msg[p];
        if ((l & 0xC0) == 0xC0) {                /* pointer: 14-bit offset */
            if (p + 1 >= len) return -1;
            if (!jumped) *pos = p + 2;
            p = ((l & 0x3F) << 8) | msg[p + 1];
            jumped = 1;
            continue;
        }
        if (l == 0) {                            /* root label: the end */
            if (!jumped) *pos = p + 1;
            break;
        }
        if (p + 1 + l > len || n + l + 2 > cap) return -1;
        memcpy(out + n, msg + p + 1, l);         /* copy one label */
        n += l;
        out[n++] = '.';
        p += 1 + l;
    }
    if (n == 0) strcpy(out, ".");                /* the root itself */
    else out[n - 1] = '\0';                      /* drop the final dot */
    return 0;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "usage: %s name [type] [server] [port]\n", argv[0]);
        return 1;
    }
    const char *qname = argv[1], *server = argc > 3 ? argv[3] : "1.1.1.1";
    int qtype = 0, port = argc > 4 ? atoi(argv[4]) : 53;
    for (int i = 0; i < NTYPES; i++)
        if (strcasecmp(argc > 2 ? argv[2] : "A", types[i].name) == 0)
            qtype = types[i].code;
    if (!qtype) { fprintf(stderr, "unknown type\n"); return 1; }

    /* --- the query: a 12-byte header, then the question;
           every number in network byte order --- */
    unsigned char q[512];
    int n = 0;
    srand((unsigned)time(NULL) ^ (unsigned)getpid());
    uint16_t id = (uint16_t)rand();
    uint16_t hdr[6] = { htons(id), htons(0x0100), /* flags: RD = 1 */
                        htons(1), 0, 0, 0 };     /* one question */
    memcpy(q, hdr, 12);
    n = 12;
    /* the name as labels: www.kernel.org -> 3 www 6 kernel 3 org 0 */
```

```
    for (const char *s = qname; *s; ) {
        const char *dot = strchr(s, '.');
        size_t l = dot ? (size_t)(dot - s) : strlen(s);
        if (l == 0 || l > 63 || n + l + 6 > sizeof q) {
            fprintf(stderr, "bad name\n");
            return 1;
        }
        q[n++] = (unsigned char)l;
        memcpy(q + n, s, l);
        n += (int)l;
        s += l + (dot ? 1 : 0);
    }
    q[n++] = 0;
    uint16_t tc[2] = { htons((uint16_t)qtype), htons(1) };  /* QCLASS IN */
    memcpy(q + n, tc, 4);
    n += 4;
    printf("query (%d bytes):", n);
    for (int i = 0; i < n; i++) printf(" %02x", q[i]);
    printf("\n");

    /* --- one UDP datagram out, one back: no connection, no handshake --- */
    int fd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in to = {0};
    to.sin_family = AF_INET;
    to.sin_port = htons((uint16_t)port);
    if (inet_pton(AF_INET, server, &to.sin_addr) != 1) {
        fprintf(stderr, "the server must be an IPv4 address\n");
        return 1;
    }
    struct timeval tv = {2, 0};                   /* wait at most 2 s */
    setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
    unsigned char r[4096];
    ssize_t len = -1;
    for (int attempt = 1; attempt <= 3 && len < 0; attempt++) {
        sendto(fd, q, (size_t)n, 0, (struct sockaddr *)&to, sizeof to);
        len = recvfrom(fd, r, sizeof r, 0, NULL, NULL);
        if (len < 0) printf("no answer after 2 s (attempt %d)\n", attempt);
    }                                             /* UDP may lose it: we retry */
    close(fd);
    if (len < 12) return 1;

    uint16_t f[6];                            /* the header, in host order */
    for (int i = 0; i < 6; i++) f[i] = get16(r + 2 * i);
    if (f[0] != id) { fprintf(stderr, "wrong id\n"); return 1; }
    printf("response (%zd bytes): id=0x%04x flags=0x%04x rcode=%d%s%s\n",
           len, f[0], f[1], f[1] & 0xF, (f[1] & 0x0400) ? " AA" : "",
           (f[1] & 0x0200) ? " TC" : "");
    printf("question=%d answer=%d authority=%d additional=%d\n",
           f[2], f[3], f[4], f[5]);

    int pos = 12;
    char name[256];
    for (int i = 0; i < f[2]; i++) {        /* skip the question(s) */
        if (read_name(r, (int)len, &pos, name, sizeof name) < 0) return 1;
        pos += 4;
    }
    for (int i = 0; i < f[3] + f[4] + f[5]; i++) {   /* resource records */
        if (read_name(r, (int)len, &pos, name, sizeof name) < 0) return 1;
        if (pos + 10 > len) return 1;
        uint16_t t = get16(r + pos), rdlen = get16(r + pos + 8);
        uint32_t ttl = (uint32_t)get16(r + pos + 4) << 16 | get16(r + pos + 6);
        pos += 10;
        if (pos + rdlen > len) return 1;
        char val[256] = "", ex[200];
        int vp = pos;
        if (t == 1 && rdlen == 4)
```

```
            inet_ntop(AF_INET, r + pos, val, sizeof val);
        else if (t == 28 && rdlen == 16)
            inet_ntop(AF_INET6, r + pos, val, sizeof val);
        else if (t == 2 || t == 5 || t == 12)       /* NS, CNAME, PTR */
            read_name(r, (int)len, &vp, val, sizeof val);
        else if (t == 15) {                         /* MX: preference, name */
            vp += 2;
            read_name(r, (int)len, &vp, ex, sizeof ex);
            snprintf(val, sizeof val, "%d %s", get16(r + pos), ex);
        } else if (t == 16)                         /* TXT: first string */
            snprintf(val, sizeof val, "\"%.*s\"", r[pos],
                     (const char *)r + pos + 1);
        else
            snprintf(val, sizeof val, "(%d bytes of data)", rdlen);
        const char *sec = i < f[3] ? "answer"
                        : i < f[3] + f[4] ? "authority" : "additional";
        printf("%-10s %-26s %6u  %-5s %s\n", sec, name, ttl,
               type_name(t), val);
        pos += rdlen;
    }
    return 0;
}
```

```
make -C linux
# terminal C: watch port 53
sudo tcpdump -i any -n -X udp port 53
# terminal B
./linux/dns_query www.unibuc.ro A 1.1.1.1
./linux/dns_query unibuc.ro MX
./linux/dns_query gmail.com TXT 8.8.8.8
./linux/dns_query www.unibuc.ro A 198.41.0.4   # a.root-servers.net: referral
python3 python/dns_query.py www.unibuc.ro AAAA            # the same in Python
```

- **Q11.** In the tcpdump output of the first query, mark the ID, the flags (0x0100: what is set?), the counts and the question bytes. Compare with your homework query for unibuc.ro MX.
- **Q12.** In the reply, find a compression pointer (`c0 ..`) and say which offset it points to and which name is there. How does `read_name` tell a pointer from a label length, and how does it avoid an endless loop on a malicious reply?
- **Q13.** Query `a.root-servers.net` (198.41.0.4; the program takes addresses only) directly — RD is ignored there. Why is ANCOUNT 0, and what do you find in the Authority and Additional sections? Extend the program: when the reply is a referral, query the first server in the Additional section, and so on until an answer arrives — your own iterative resolver. (Hint: put the loop around `sendto`/`recvfrom`; keep the A records of the Additional section.)

Task 6 — A NAT router of your own

`nat_lab.sh` builds three network namespaces — isolated network stacks inside your machine — connected by virtual Ethernet cables: a home network `lan`, a `router` with a private and a public interface, and an Internet `server`. The router forwards packets and masquerades the LAN behind its public address with one nftables rule. The server has no route to the private network, as on the real Internet.

```
#!/bin/sh
# nat_lab.sh - a home network, a NAT router and a server inside one Linux
# machine, built from network namespaces (Lab 5).
# usage: sudo ./nat_lab.sh up | down
#
#  [lan] 192.168.77.2 --- 192.168.77.1 [router]               (private side)
#                         203.0.113.1  [router] --- 203.0.113.10 [server]
#                         the router masquerades: NAT       (public side)
set -e
```

```
if [ "$1" = "down" ]; then
    for ns in lan router server; do ip netns del $ns 2>/dev/null || true; done
    echo "removed"; exit 0
fi
[ "$1" = "up" ] || { echo "usage: $0 up | down"; exit 1; }

for ns in lan router server; do
    ip netns add $ns; ip -n $ns link set lo up
done
# two virtual cables: lan <-> router and router <-> server
ip link add veth-lan netns lan type veth peer name veth-in netns router
ip link add veth-out netns router type veth peer name veth-srv netns server

ip -n lan    addr add 192.168.77.2/24 dev veth-lan
ip -n router addr add 192.168.77.1/24 dev veth-in
ip -n router addr add 203.0.113.1/24  dev veth-out
ip -n server addr add 203.0.113.10/24 dev veth-srv
ip -n lan    link set veth-lan up
ip -n router link set veth-in up
ip -n router link set veth-out up
ip -n server link set veth-srv up

ip -n lan route add default via 192.168.77.1     # the lan's default gateway
ip netns exec router sysctl -q -w net.ipv4.ip_forward=1
# The server has no route back to 192.168.77.0/24: without NAT its
# replies could not return.

# NAT: packets leaving towards the public side get the router's address
ip netns exec router nft -f - <<'NFT'
table ip nat {
    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;
        oifname "veth-out" ip saddr 192.168.77.0/24 masquerade
    }
}
NFT
echo "ready: lan 192.168.77.2 -> router (NAT) -> server 203.0.113.10"
```

```
chmod +x linux/nat_lab.sh; sudo ./linux/nat_lab.sh up
# terminal C: a web server on the 'Internet'
sudo ip netns exec server python3 -m http.server 80 --bind 203.0.113.10
# terminal B: a client in the home network
sudo ip netns exec lan curl -sI http://203.0.113.10/ | head -1
sudo ip netns exec router conntrack -L          # the translation table
sudo ip netns exec router nft list ruleset
```

Remove NAT and try again, then rebuild the lab and add **port forwarding**: a web server inside the home network becomes reachable from outside on the router's port 8080.

```
sudo ip netns exec router nft flush table ip nat
sudo ip netns exec lan curl -m 3 http://203.0.113.10/     # times out: why?
sudo ./linux/nat_lab.sh down; sudo ./linux/nat_lab.sh up
sudo ip netns exec lan python3 -m http.server 8080        # terminal C
sudo ip netns exec router nft add chain ip nat prerouting \
    '{ type nat hook prerouting priority dstnat; policy accept; }'
sudo ip netns exec router nft add rule ip nat prerouting \
    iifname veth-out tcp dport 8080 dnat to 192.168.77.2:8080
sudo ip netns exec server curl -s http://203.0.113.1:8080/ | head -3
sudo ./linux/nat_lab.sh down                              # at the end
```

Finally, find the mapping that the NATs of your real network create. A STUN server reports the address and port from which your UDP packet arrived:

```
"""stun_client.py - ask a STUN server which public address and port your
UDP packets come from, i.e. which mapping your NAT created (Lab 5).
usage: python3 stun_client.py [server] [port]
```

```
       (default stun.l.google.com 19302; RFC 8489 Binding request)"""
import os, socket, struct, sys

server = sys.argv[1] if len(sys.argv) > 1 else "stun.l.google.com"
port = int(sys.argv[2]) if len(sys.argv) > 2 else 19302
MAGIC = 0x2112A442

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.settimeout(3)
sock.bind(("0.0.0.0", 0))
print("local (private) side:", sock.getsockname())
for attempt in range(2):                  # two requests from the same socket
    tid = os.urandom(12)                  # transaction id
    request = struct.pack("!HHI", 0x0001, 0, MAGIC) + tid   # Binding request
    sock.sendto(request, (server, port))
    data, src = sock.recvfrom(2048)
    mtype, length, magic = struct.unpack("!HHI", data[:8])
    assert data[8:20] == tid and mtype == 0x0101, "not our Binding response"
    pos = 20
    while pos < 20 + length:              # attributes: type, length, value
        atype, alen = struct.unpack("!HH", data[pos:pos + 4])
        value = data[pos + 4:pos + 4 + alen]
        if atype == 0x0020:               # XOR-MAPPED-ADDRESS
            xport = struct.unpack("!H", value[2:4])[0] ^ (MAGIC >> 16)
            xaddr = struct.unpack("!I", value[4:8])[0] ^ MAGIC
            print(f"request {attempt + 1}: the server saw us as "
                  f"{socket.inet_ntoa(struct.pack('!I', xaddr))}:{xport}")
        pos += 4 + (alen + 3) // 4 * 4    # values are padded to 4 bytes
```

```
ip -4 addr show | grep inet                       # your private side
python3 python/stun_client.py                     # your public side
```

- **Q14.** Which client address did the web server log: 192.168.77.2 or 203.0.113.1? Copy the conntrack entry and explain its two halves (original and reply direction). Why did the request time out after `flush`?
- **Q15.** With port forwarding, which client address did the web server in `lan` log? Which nftables chain and hook did each rule use, and why must DNAT happen *before* routing and masquerading *after* it?
- **Q16.** Compare your private address with the address reported by STUN. Is your machine behind NAT? Behind CGNAT (is the router's WAN address in 100.64.0.0/10)? Did both requests get the same public port — what does this say about the NAT (endpoint-independent mapping)?

If something does not work

Symptom	Likely cause and what to try
`No module named aiosmtpd`	install `python3-aiosmtpd` (Ubuntu universe) or `pip install --user aiosmtpd`
talk.py ends before the last reply	it waits 5 s after your last line; press Ctrl-C when done, or use `nc -C`
`dig: command not found`	install `dnsutils` (Debian/Ubuntu) or `bind-utils` (Fedora)
`dig +trace` or Task 5 times out	the network blocks outgoing DNS to servers other than its own resolver: use the resolver in `/etc/resolv.conf` (or `resolvectl status`) as the server argument
tcpdump shows queries to 127.0.0.53 only	systemd-resolved is the local stub resolver; capture with `-i any` and query an external server explicitly

Symptom	Likely cause and what to try
`nat_lab.sh`: File exists	the namespaces are left from an earlier run: `sudo ./linux/nat_lab.sh down` first
`conntrack: command not found` or empty table	install `conntrack`; list it right after the curl, before the entry expires
STUN times out	outgoing UDP is blocked on this network (some campus and corporate firewalls): try from home or a mobile hotspot

PART D · WEEK 5

Laboratory 5 (Windows): the same protocols with Windows tools and Winsock; finding your own NAT

Two-hour laboratory sheet (2 × 50 minutes) accompanying Lecture 5

This sheet follows the Linux sheet task by task, so that answers can be compared. The mail servers are the same Python programs; DNS is explored with PowerShell's `Resolve-DnsName`, `nslookup` and Windows' own DNS cache; the DNS client is the Winsock version of the C program. Network namespaces do not exist on Windows, so instead of building a NAT router you find the NATs between your PC and the Internet — and the one Windows itself runs for WSL.

By the end of the laboratory you should be able to

- send a message by typing SMTP, and a MIME message with an attachment;
- read and delete mail with POP3 by hand, and open an IMAP session over TLS;
- query DNS with Resolve-DnsName and nslookup, and inspect and clear Windows' DNS cache;
- build and extend a DNS client with Winsock and compare its packets in Wireshark;
- find out how many NATs separate your PC from the Internet, and which public mapping they create.

Before you start

- **Python 3** from python.org (tick "Add to PATH"); then `py -m pip install aiosmtpd`.
- **A C compiler**, as in Laboratories 3 and 4: MinGW-w64 (`gcc ... -lws2_32`) or Visual Studio Build Tools (`cl ... ws2_32.lib`).
- **Wireshark** with Npcap (with loopback capture support).
- **Files.** Unpack the archive into `lab5\`; the Windows folder contains `dns_query_win.c`; the Python programs are in `python\`. Work in PowerShell from `lab5\`.
- **What to hand in.** A short report with answers to **Q1–Q16** and the outputs that support them; your extended `dns_query_win.c`.

Plan

Step	Activity	Time
1	SMTP by hand: a mail server of your own	15 min
2	MIME: an attachment on the wire	10 min
3	Reading mail: POP3 by hand, an IMAP greeting	15 min
	Break	10 min
4	DNS with Resolve-DnsName, nslookup and the Windows cache	15 min
5	The DNS client with Winsock	20 min
6	Finding your own NAT; STUN	15 min

Task 1 — SMTP by hand

```
# window A: an SMTP server on port 8025, storing mail in .\Maildir
py -m aiosmtpd -n -l 127.0.0.1:8025 -c aiosmtpd.handlers.Mailbox Maildir
# window B: speak SMTP (type the lines of the Linux sheet, Task 1)
py python\talk.py 127.0.0.1 8025
Get-ChildItem Maildir\new; Get-Content (Get-ChildItem Maildir\new)[0].FullName
```

Type EHLO `client.lab.test`, MAIL FROM:`<alice@lab.test>`, RCPT TO:`<bob@lab.test>`, RCPT TO:`<carol@lab.test>`, DATA, three header lines, an empty line, a body containing a line that starts with two dots, a line with a single dot, and QUIT — as in the Linux sheet. With the Windows TELNET client installed, `telnet 127.0.0.1 8025` works too, but it sends each key as you type it.

- **Q1.** Copy the server's reply to EHLO. Which reply codes did you get for MAIL, RCPT, DATA, the final dot and QUIT, and what class is each?
- **Q2.** Carol is not named in any header line. Does she receive the message? Where does the envelope appear in the stored file?
- **Q3.** How does the line you typed with two dots appear in the stored message, and why?

Task 2 — MIME: an attachment on the wire

`send_mail.py` is listed in the Linux sheet; it builds the message with Python's `email` package and prints the SMTP dialogue.

```
$b = New-Object byte[] 30000; (New-Object Random).NextBytes($b)
[IO.File]::WriteAllBytes("$PWD\photo.bin", $b)
py python\send_mail.py 127.0.0.1 8025 photo.bin
Get-ChildItem photo.bin, Maildir\new | Format-Table Name, Length
notepad (Get-ChildItem Maildir\new | Sort-Object LastWriteTime)[-1].FullName
```

- **Q4.** Identify the envelope and the start of the message in the debug output. How was the subject's dash written? Decode it: `[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String('4oCU'))`.
- **Q5.** Find the `Content-Type` of the message and of each part, the boundary and the transfer encoding of the attachment. By what percentage is the stored message larger than `photo.bin`?

Task 3 — Reading mail: POP3 and IMAP

```
py python\pop3_server.py Maildir 1110          # window C
py python\talk.py 127.0.0.1 1110               # window B
py python\talk.py imap.gmail.com 993 --tls     # then: a1 CAPABILITY, a2 LOGOUT
```

In the POP3 session type USER `bob`, PASS `secret`, STAT, LIST, UIDL, TOP `1 0`, RETR `1`, DELE `1`, RSET, DELE `1`, QUIT. Capture the session in Wireshark on the loopback adapter with the filter `tcp.port == 1110`, and the IMAP session on your network adapter with `tcp.port == 993`.

- **Q6.** Explain the replies to STAT and LIST. How does RETR mark the end of the message? Is the POP3 password readable in the capture? Which messages remain in `Maildir\new` after QUIT?
- **Q7.** Copy the IMAP greeting and the CAPABILITY line; find `IMAP4rev1`, `IDLE` and the AUTH= methods. What does Wireshark show of the IMAP session? Which server name can you still read, and in which TLS message?

Task 4 — DNS with Windows tools

```
Resolve-DnsName www.unibuc.ro                       # type A by default
Resolve-DnsName unibuc.ro -Type MX
Resolve-DnsName unibuc.ro -Type NS
Resolve-DnsName gmail.com -Type TXT -Server 1.1.1.1
Resolve-DnsName _dmarc.gmail.com -Type TXT
Resolve-DnsName 8.8.8.8                             # reverse: PTR
Resolve-DnsName www.unibuc.ro -Server a.root-servers.net -NoRecursion
nslookup -type=soa unibuc.ro
nslookup -debug www.google.com 1.1.1.1              # header and TTLs
ipconfig /displaydns | Select-String -Context 0,6 'unibuc'
ipconfig /flushdns                                  # (admin) empty the cache
Get-DnsClientServerAddress -AddressFamily IPv4      # your resolvers
```

- **Q8.** Which resolver does your PC use? Run `nslookup -debug www.google.com 1.1.1.1` twice, a few seconds apart: how did the TTL change, and why?
- **Q9.** Query `a.root-servers.net` with `-NoRecursion`. Which servers does the reply name, in which section, and why does it also contain addresses? Continue by hand to the authoritative answer, as in homework 2 of the seminar.
- **Q10.** Find www.unibuc.ro in `ipconfig /displaydns`: which TTL does Windows' cache show? What are the SPF and DMARC records of gmail.com, and what do they ask a receiving server to do?

Task 5 — The DNS client with Winsock

`dns_query_win.c` is the Linux program of Laboratory 5 with the Winsock changes you know: `WSAStartup`, `SOCKET`, `closesocket`, and a receive timeout given as a `DWORD` in milliseconds instead of a `struct timeval`. Read the full listing in the Linux sheet; the lines that differ are:

```
#include <winsock2.h>
#include <ws2tcpip.h>
...
WSADATA wsa;
if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) return 1;
SOCKET fd = socket(AF_INET, SOCK_DGRAM, 0);
DWORD tv = 2000;                 /* Winsock: milliseconds, not timeval */
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, (const char *)&tv, sizeof tv);
...
closesocket(fd);
WSACleanup();

gcc -Wall -O2 windows\dns_query_win.c -o dns_query.exe -lws2_32
# or: cl windows\dns_query_win.c ws2_32.lib
.\dns_query.exe www.unibuc.ro A 1.1.1.1
.\dns_query.exe unibuc.ro MX
.\dns_query.exe www.unibuc.ro A 198.41.0.4      # a.root-servers.net
py python\dns_query.py www.unibuc.ro AAAA
```

Capture on your network adapter with the Wireshark display filter `dns`. Wireshark decodes every field; compare its tree with your program's output and with the bytes in the lower pane.

- **Q11.** In Wireshark, mark the ID, the flags, the counts and the question bytes of your query. Is the reply matched to the query? By which field?
- **Q12.** Find a compression pointer in the reply (`c0 ..`): which offset does it point to, and which name is there? How does `read_name` tell a pointer from a label length?
- **Q13.** Why is ANCOUNT 0 in the reply of `a.root-servers.net`? Extend the program into an iterative resolver that follows referrals, as described in Q13 of the Linux sheet.

Task 6 — Finding your own NAT

```
ipconfig | Select-String 'IPv4|Gateway'            # your private side
curl.exe -s https://ifconfig.me; ''                # the address servers see
tracert -d -h 4 1.1.1.1                            # the first routers
py python\stun_client.py                          # the UDP mapping
py python\stun_client.py stun.cloudflare.com 3478
```

Open your home router's status page (usually at the gateway address) and find its WAN (Internet) address. Compare the three addresses: your PC's, the router's WAN address, and the one that `ifconfig.me` and STUN report. If the router's WAN address is private or in **100.64.0.0/10**, your ISP runs carrier-grade NAT, and there are two NATs on your path.

Windows runs a NAT of its own: **WSL 2** and Hyper-V's *Default Switch* give the Linux virtual machine a private address behind a NAT inside Windows. If you have WSL 2, compare `ip -4 addr` inside WSL with `ipconfig` in Windows (the *vEthernet (WSL)* adapter), and run `curl ifconfig.me` in both.

- **Q14.** Write your PC's address, the gateway, the router's WAN address and the public address reported by ifconfig.me. Which are private (RFC 1918 or 100.64.0.0/10)? How many NATs are on your path?
- **Q15.** Does STUN report the same address as ifconfig.me? Did both STUN requests from the same socket see the same public port? Was the port the same as the local one? What does this say about your NAT?
- **Q16.** (WSL 2) Draw the path of a packet from WSL to ifconfig.me, with every address and every translation. Could a program in WSL be reached from another PC on your LAN without configuration? (Windows calls the fix `netsh interface portproxy` — a port forwarding rule.)

If something does not work

Symptom	Likely cause and what to try
`'py' is not recognized`	Python is not installed or not on PATH; reinstall from python.org with "Add to PATH", or use `python`
aiosmtpd: `address already in use`	another server uses port 8025: choose another port in both windows
talk.py ends before the last reply	it waits 5 s after the last line; press Ctrl-C when done
`Resolve-DnsName` to an external server times out	the network allows DNS only to its own resolver: use the address from `Get-DnsClientServerAddress`
Wireshark shows no DNS packets	the answer came from Windows' cache (`ipconfig /flushdns`), or the browser uses DNS over HTTPS
`curl` prints an object instead of text	you typed the PowerShell alias: always type `curl.exe`
STUN times out	outgoing UDP is blocked on this network; try from home or a mobile hotspot

Next week

With Lecture 5 the application layer is complete: every topic of the syllabus list — FTP, HTTP, HTTPS, SSH, SMTP, POP, IMAP, TELNET, Usenet, DNS and NAT — has been covered in Weeks 3–5. Lecture 6 moves down to the **transport layer**: how port numbers multiplex many applications over one

address, UDP, and the principles of reliable data transfer that TCP is built on. In Laboratory 6 you write a reliable transfer protocol of your own over UDP and watch it recover from lost packets.